Smart Courier Planner

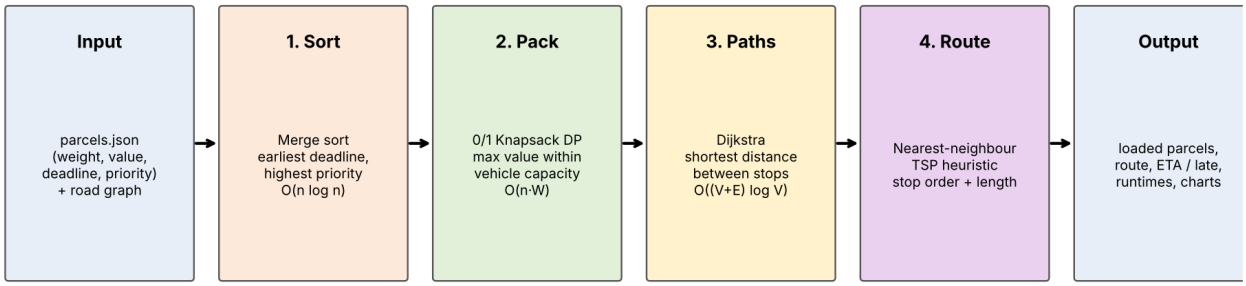
Capstone Project: Sorting, Dynamic Programming, Graph Algorithms and Approximation

Name: _____ Student ID: _____ GitHub: https://github.com/<you>/courier_planner

This report describes an integrated Python prototype that, given parcels and delivery points, produces a prioritised packing plan and an ordered delivery route. It combines merge sort, 0/1 knapsack dynamic programming, Dijkstra's algorithm and a nearest-neighbour TSP approximation, and compares the runtime of each component.

Phase 1: Courier Problem Design and Planning

Problem. A courier vehicle with limited capacity leaves a depot. Many parcels are waiting, each with a destination, weight, value, deadline and priority. The system must (1) rank parcels by urgency, (2) pick the best set that fits in the vehicle, (3) compute shortest distances over the road network and (4) order the stops into a short tour.



Selected parcels' destinations become the delivery points that Dijkstra and the TSP heuristic work on.

Parcel attributes

Attribute	Meaning	Used by
parcel_id	Unique identifier	all stages
destination	Graph node where it is delivered	routing
weight (kg, int)	Space it consumes in the vehicle	knapsack
value	Benefit score of delivering it now	knapsack
deadline (h)	Latest acceptable delivery time	sorting, on-time check
priority (1-3)	Tie-breaker between equal deadlines	sorting

Graph format

An undirected weighted graph stored as an adjacency dictionary `{node: {neighbour: km}}`. Datasets are JSON files with `nodes` (coordinates, used only for drawing), `edges` (u, v, km), `parcels`, the `depot` and the `vehicle_capacity`. Datasets are validated on load (unknown nodes, duplicate ids, bad weights).

Algorithm selection

Task	Algorithm	Why it fits	Complexity
Prioritise parcels	Merge sort	Stable, guaranteed O(n log n); ties keep input order	O(n log n)
Choose load	0/1 knapsack DP	Exact optimum; parcels cannot be split	O(n·W)
Shortest paths	Dijkstra + heap	Non-negative road lengths	O((V+E) log V)
Order stops	Nearest neighbour	TSP is NP-hard; fast heuristic	O(k·(V+E) log V)

Phase 2: Package Prioritization and Box Filling

Sorting. Parcels are sorted with a stable merge sort using the key (deadline, -priority, weight): earliest deadline first, then highest priority, then lightest. Quick sort and insertion sort are also implemented so the choice can be justified experimentally in Phase 4 (merge sort was about 1.5x faster than quick sort and far faster than insertion sort at 2000 parcels).

Dynamic programming. Let `T[i][w]` be the best value using the first i sorted parcels with capacity w. `T[i][w] = max(T[i-1][w], T[i-1][w-wt_i] + val_i)` when the parcel fits, otherwise `T[i-1][w]`. Back-tracking from `T[n][W]` recovers the chosen parcels; on equal values the more urgent parcel is preferred.

Result (small dataset, capacity 20 kg). Selected parcels: **P001, P003, P004**; total weight 20/20 kg; total value **120**. Output trace (sorted list, selection, route):

```
Dataset: small | parcels: 10 | nodes: 8 | capacity: 20 kg
======================================================================
TRIP 1  (departs at t = 0.00 h)
======================================================================

[1] Parcels sorted by urgency (deadline -> priority -> weight)
    ID     Dest     Wt  Value  Deadline  Prio
    P001   A         5     40       2.0     3
    P003   C         6     35       2.5     3
    P002   B         8     30       3.0     2
    P008   C        10     50       3.5     3
    P004   D         9     45       4.0     2
    P010   B         6     22       4.5     2
    P005   E         4     15       5.0     1
    P006   F         7     28       6.0     2
    P009   D         2     10       6.0     1
    P007   A         3     12       8.0     1

[2] Knapsack selection (capacity 20 kg)
    Loaded : P001, P003, P004
    Weight : 20 / 20 kg
    Value  : 120

[3] Delivery route (Dijkstra legs + nearest-neighbour order)
    Stop order : Depot -> A -> C -> D -> Depot
     Depot -> A         4.0 km   via Depot > A
         A -> C         6.0 km   via A > C
         C -> D         3.0 km   via C > D
         D -> Depot    11.0 km   via D > C > J > Depot
    Full path  : Depot > A > C > D > C > J > Depot
    Route length: 24.0 km   Duration: 1.50 h

[4] Expected deliveries
    ID     Dest     ETA (h)  Deadline  Status
    P001   A           0.20       2.0  on time
    P003   C           0.60       2.5  on time
    P004   D           0.85       4.0  on time

[5] Stage runtimes
    sorting        0.017 ms
    knapsack       0.034 ms
    routing        0.052 ms
```

DP table evidence (rows = parcels in sorted order, columns = capacity 0-20; bottom-right cell = 120 = optimum):

```
[6] Knapsack DP table
Parcel      0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
-------------------------------------------------------------------------------------------
(none)      0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
P001        0   0   0   0   0  40  40  40  40  40  40  40  40  40  40  40  40  40  40  40  40
P003        0   0   0   0   0  40  40  40  40  40  40  75  75  75  75  75  75  75  75  75  75
P002        0   0   0   0   0  40  40  40  40  40  40  75  75  75  75  75  75  75  75 105 105
P008        0   0   0   0   0  40  40  40  40  40  50  75  75  75  75  90  90  90  90 105 105
P004        0   0   0   0   0  40  40  40  40  45  50  75  75  75  85  90  90  90  90 105 120
P010        0   0   0   0   0  40  40  40  40  45  50  75  75  75  85  90  90  97  97 105 120
P005        0   0   0   0  15  40  40  40  40  55  55  75  75  75  85  90  90  97 100 105 120
P006        0   0   0   0  15  40  40  40  40  55  55  75  75  75  85  90  90  97 103 105 120
P009        0   0  10  10  15  40  40  50  50  55  55  75  75  85  85  90  95 100 103 107 120
P007        0   0  10  12  15  40  40  50  52  55  62  75  75  85  87  90  97 100 103 107 120
```

Phase 3: Route Maker and Graph Processing

Customer graph. The depot, six delivery points (A-F) and one junction (J) are connected by 12 roads. Delivery points are not all directly connected, so the distance between two stops is the Dijkstra shortest-path distance (for example D to Depot is 11 km via D > C > J > Depot, shorter than any direct alternative).

Nearest-neighbour TSP. Starting at the depot, the vehicle repeatedly drives to the closest unvisited stop (distance from Dijkstra), then returns to the depot. Ties are broken by name so the result is deterministic.

Leg	Distance (km)	Shortest path
Depot to A	4.0	Depot > A
A to C	6.0	A > C
C to D	3.0	C > D
D to Depot	11.0	D > C > J > Depot
Total	24.0	Depot > A > C > D > C > J > Depot

Expected arrival times use 20 km/h and 0.1 h service per stop: P001 at 0.20 h (deadline 2.0), P003 at 0.60 h (2.5), P004 at 0.85 h (4.0); all on time. Total trip duration 1.50 h.

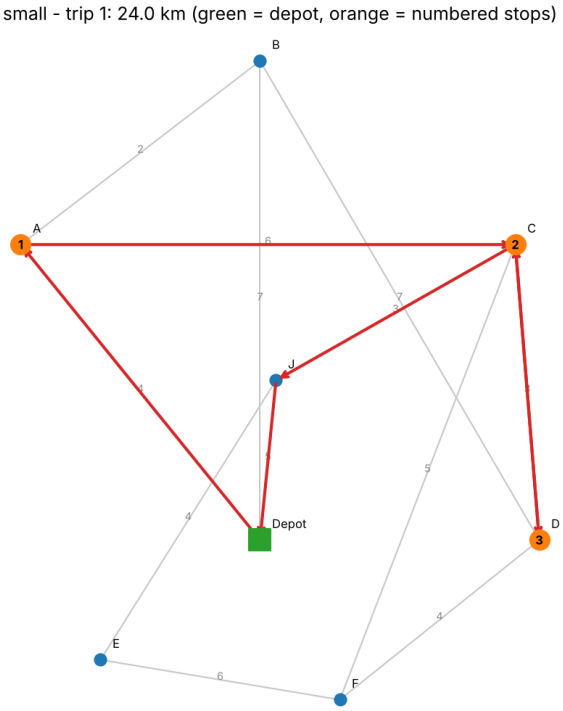
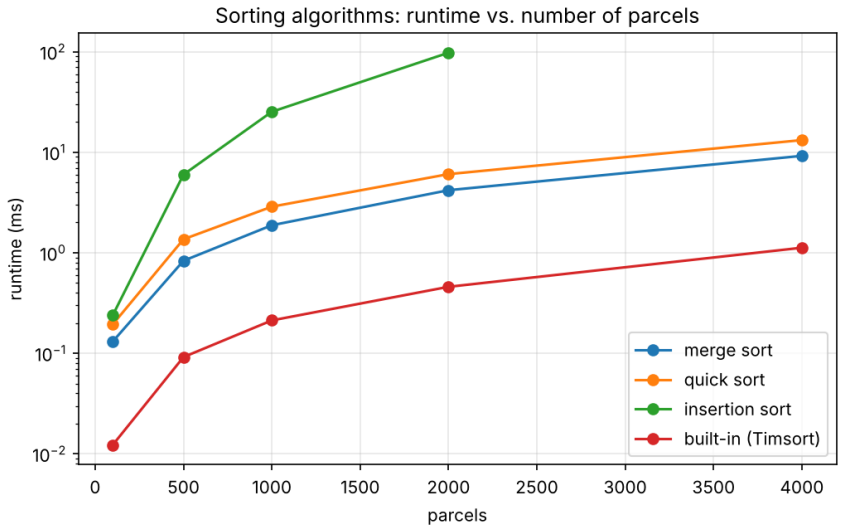


Figure: final route. Green square = depot, numbered orange nodes = stops in visiting order.

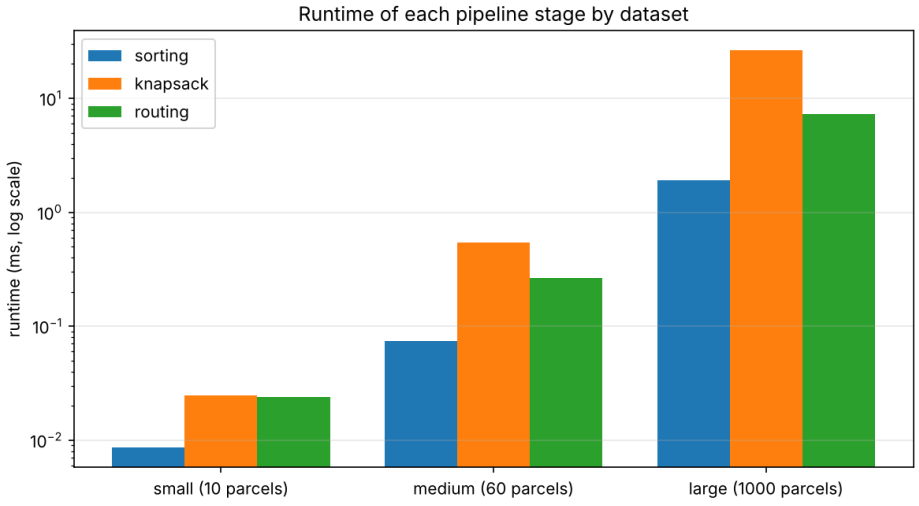
Phase 4: Performance Testing and Optimization

Test cases. 18 unit tests (`python -m unittest discover -s tests -t .`) check all sorting algorithms against Python's sorted(), the knapsack against brute force on 30 random instances, Dijkstra on a hand-checked graph (including unreachable and invalid nodes), route validity, capacity limits, and the known result of the small dataset. All pass.

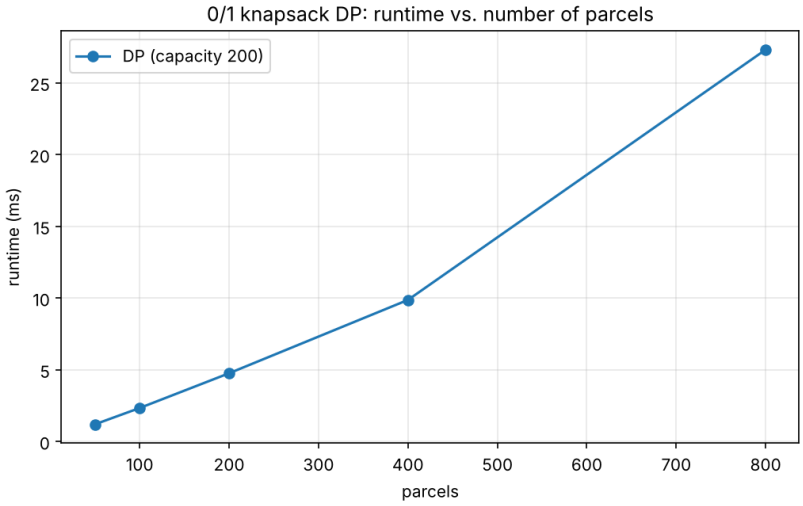
Datasets. small (10 parcels, 8 nodes), medium (60 parcels, 30 nodes) and large (1000 parcels, 150 nodes), the last two generated reproducibly with fixed random seeds. Each measurement is the best of 3 runs.

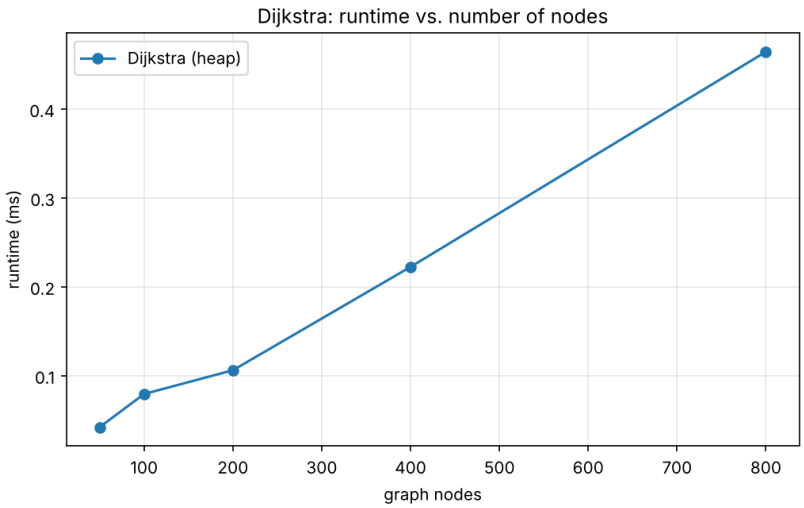


Merge sort and quick sort grow almost linearly (O(n log n)); insertion sort grows quadratically (about 97 ms at 2000 parcels vs 4 ms for merge sort). Built-in Timsort is fastest because it is C code.



Stage runtimes on the three datasets (log scale). On the large dataset the knapsack dominates (about 26 ms), then routing (about 7 ms), then sorting (about 2 ms).





The DP runtime grows linearly with the number of parcels for a fixed capacity (O(n·W)); Dijkstra grows slightly more than linearly with the number of nodes, as expected for O((V+E) log V).

Observed runtimes (ms)

Experiment	Algorithm	Size	ms
sorting	merge sort	1000	1.879
sorting	quick sort	1000	2.874
sorting	insertion sort	1000	25.273
sorting	built-in (Timsort)	1000	0.212
sorting	merge sort	2000	4.192
sorting	quick sort	2000	6.059
sorting	insertion sort	2000	97.686
sorting	built-in (Timsort)	2000	0.459
knapsack	DP (capacity 200)	100	2.332
knapsack	DP (capacity 200)	800	27.307
dijkstra	Dijkstra (heap)	100	0.079
dijkstra	Dijkstra (heap)	800	0.464

Suggested optimizations

- **Knapsack memory/time:** use a 1-D table (O(W) memory) when the full table is not needed as evidence, or NumPy vectorisation to speed up the inner loop.
- **Dijkstra:** cache distances between stops across trips, or precompute all-pairs distances once for repeated planning.
- **Route quality:** apply 2-opt improvement on top of nearest neighbour to remove crossing legs.
- **Deadlines:** the knapsack uses a fixed value and ignores route length, so far-away parcels can be late (visible on the medium and large datasets). Including deadline penalties in the objective, or re-sorting stops by deadline, would address this.

Phase 5: Final Report and Demonstration

Demonstration. Run `python main.py` for the small scenario, or `python main.py --dataset data/medium.json --plot results/route_medium.png`. Output trace for the medium dataset (60 parcels, 80 kg vehicle), abbreviated:

```
Dataset: medium | parcels: 60 | nodes: 30 | capacity: 80 kg
[3] Delivery route (Dijkstra legs + nearest-neighbour order)
    Stop order : Depot -> N008 -> N007 -> N025 -> N021 -> N013 -> N004 -> N017 -> N015 -> N010 - ...
     Depot -> N008     13.6 km   via Depot > N008
      N008 -> N007     31.0 km   via N008 > N007
      N007 -> N025     19.5 km   via N007 > N025
      N025 -> N021     32.2 km   via N025 > N016 > N012 > N021
    ...
    Route length: 530.8 km   Duration: 27.94 h
```

Explanation of results. On the small dataset the DP loads P001, P003 and P004. Although P008 has the highest single value (50), taking P001 + P003 + P004 (value 120, 20 kg) beats every other combination that fits, which a greedy choice by value would miss. The three loaded stops are A, C and D; Dijkstra finds the 11 km return from D through C and J, and the 24 km tour meets every deadline.

Limitations. The route heuristic is not optimal; the knapsack ignores route length; speed is a constant average; the road network is undirected with non-negative weights.

How the repository is organised.

- `courier/`: one module per algorithm plus planner, dataset and report helpers.
- `tests/`, `data/`, `results/`: unit tests, sample datasets, saved outputs and charts.
- `README.md`: installation, execution steps, dataset format, design decisions.

Originality. The implementation was written for this capstone; all algorithms are coded from scratch (no routing or optimisation libraries). The only third-party package is matplotlib, used for charts.